Chapter 7 worked solutions

DS07-E01 • Stability and inversion count

Passes are `(2,B),(5,A),(4,C),(2,D)`; `(2,B),(4,C),(5,A),(2,D)`; and `(2,B),(2,D),(4,C),(5,A)`. Strict inversions are A-B, A-C, A-D, C-D, totaling 4. Shifts are 1+1+2=4. Equal keys B and D do not form a strict inversion. With $\geq$ movement, D passes B, breaking stability. Shift count then no longer equals the strict inversion count.

DS07-E02 • Three-way partition and stack depth

Initially lt=i=0, gt=6. Reading 4 advances i. Reading 2 swaps it to index 0 and gives lt=1, i=2. Reading 4 advances i=3. Reading 7 decrements gt to 5 and swaps indices 3 and 5, giving `[2,4,4,4,1,7]`, with i unchanged. Reading that 4 advances i. Reading 1 swaps it with lt=1, giving `[2,1,4,4,4,7]`, lt=2, i=gt=5. Partition is complete. Each actual recursive child is at most half the current nonequal region, so active sizes decay geometrically and stack depth is O(log n). Extreme splits still scan lengths n, n−1,… through the looped larger side, totaling $\Theta(n^2)$.

DS07-E03 • The model behind the lower bound

Distinct keys have n! relative permutations requiring separate leaves. A binary comparison tree needs height at least $\log_2(n!)$. The inequality `n!≥(n/2)^(n/2)` gives Ω(n log n). Counting sort indexes directly by key and exceeds the comparison-only information model. Equal keys do not require distinguishing n! key orders, so this argument does not force n log n work on that single input. The lower bound remains a worst-case result for the general problem.

DS07-E04 • A counterexample to unstable LSD passes

The stable units pass yields `31,11,22,14`. The tens pass must retain 11 before 14 and produce `11,14,22,31`. Reversing equal-tens order gives `14,11,22,31`, which is not sorted. With fixed key width w, `d≈ceil(w/log₂b)`. Larger b reduces passes but increases counter initialization and storage per pass. Total time is Θ(d(n+b)). The useful base depends on n, key width, and memory constraints, not only the number of passes.

DS07-E05 • Compare three partition contracts

Hoare begins i=−1,j=6. The scans stop at i=0,j=5 and swap equal 4s; then at i=2,j=4 they swap 4 and 1, producing [4,2,1,7,4,4]. Next i=3,j=2, so j=2 is returned. Recurse on [0,3) and [3,6). j is a boundary, so index 2 must not be skipped.

Lomuto scans j=0..4. The first three keys are ≤ 4; 7 stays. Processing 1 swaps it with 7 to get [4,2,4,1,7,4], then swapping the pivot with 7 gives [4,2,4,1,4,7], q=4. Recurse on [0,4),[5,6). Three-way partition returns [2,1 | 4,4,4 | 7], lt=2, gt=5; recurse on [0,2),[5,6).

DS07-E06 • Equal keys and termination

Hoare scans i=0,j=4 and swaps equal keys, then i=1,j=3. Finally i=2,j=2, so it returns j=2 and subproblem sizes 3 and 2. Lomuto sends the first four keys left and returns q=4, giving sizes 4 and 0; repeating on all-equal input takes $\Theta(n^2)$. Three-way partition gets lt=0,gt=5 in one scan and both recursive ranges are empty.

DS07-E07 • Adjacent swaps and stability

The inversions are (3,1), (3,2), (3,0), (1,0), (2,0), totaling 5. Passes are [1,2,0,3], [1,0,2,3], [0,1,2,3], with 5 swaps. Selection sort swaps 2A and 1 first to produce [1,2B,2A], reversing equal-key A and B. The ordinary swap version is unstable.

DS07-E08 • Shell groups and stability

For gap 3 the index classes (0,3), (1,4), (2,5) sort to [7,5,2,8,6,3]. Gap 1 insertion-sorts the whole array to [2,3,5,6,7,8]. In the second example gap 2 compares positions 0 and 2, yielding [2,5B,5A,8]. The final gap-1 pass sees an already key-sorted sequence and does not restore A before B, proving instability.

DS07-E09 • Winner-tree merge

Initial candidates are [2,4,1,3]. Emit 1, replace it by 7, yielding [2,4,7,3] with winner 2. Emit 2, replace by 9, yielding [9,4,7,3] with winner 3. Emit 3, replace by 6, yielding [9,4,7,6] with winner 4. A valid maximum key cannot distinguish actual data from exhaustion, so store a separate exhausted state.

DS07-E10 • Stable merge and comparison bound

Initially (i,j)=(0,0). Since 2A=2B, choose left: (1,0), [2A]. Compare 5>2B, choose right: (1,1), [2A,2B]. Compare 5>4, choose right: (1,2), [2A,2B,4]. The right run is exhausted, so append 5. Each comparison consumes one record; the last such comparison exhausts at least one run. At most m+n−1 comparisons are possible, and interleaved runs attain this bound.

References and attribution

Lin Cing, handwritten data structures notes; MIT 6.006 sorting and comparison lower bounds; ISO C++ sorting requirements.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.
https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study